

**Textul prezentării disertației (calibrat la 8-10 minute)**

## **Slide 2 (Cuprins)**

**În această prezentare voi aborda:**

**În introducere, contextul și motivația lucrării**

**Fundamentele teoretice: demonstrațiile cu cunoștințe zero și schema Schnorr**

**Tehnologiile utilizate**

**Arhitectura generală a sistemului**

**Implementarea fluxurilor de autentificare și maparea peste OAuth 2.0**

**Proprietățile de securitate**

**Rezultatele experimentale și simularea atacurilor**

**La final, concluziile, limitările și direcțiile de dezvoltare, urmate de bibliografie**

## **Slide 3 (Introducere)**

**Tema pornește de la o problemă a aplicațiilor web moderne: autentificarea se bazează pe transmiterea unui secret partajat către server. Deși HTTPS reduce riscul interceptării, arhitectura clasică rămâne vulnerabilă în fața compromiterii bazelor de date, a reutilizării parolelor și a atacurilor de tip replay.**

**Standardul OAuth 2.0, adoptat pe scară largă, nu soluționează în mod intrinsec această problemă, faza de autentificare continuând să expună credențialele în tranzit.**

**Scopul este proiectarea unui protocol hibrid care îmbină schema Schnorr, bazată pe demonstrații cu cunoștințe zero, cu cadrul OAuth 2.0, astfel încât parola să nu părăsească niciodată dispozitivul clientului. Au fost implementate și trei fluxuri OAuth clasice ca bază de referință comparativă.**

## **Slide 4 (Zero-Knowledge Proof)**

**Demonstrația cu cunoștințe zero este un protocol criptografic prin care o entitate poate dovedi cunoașterea unui secret fără a dezvălui nicio informație suplimentară. Sistemul nu urmărește să afle parola, ci doar să obțină certitudinea matematică a cunoașterii acesteia.**

**Principiul fundamental impune ca parola să nu fie transmisă prin rețea, nici în clar, nici criptată. Rețeaua transportă exclusiv valori matematice efemere, care nu pot fi reutilizate și din care secretul nu poate fi extras.**

Astfel, protocolul oferă reziliență la interceptare, previne atacurile replay și atenuează impactul breșelor de date, serverul stocând exclusiv chei publice.

#### Slide 5 (Schema Schnorr)

Schema Schnorr se fundamentează pe dificultatea problemei logaritmului discret. Se utilizează un număr prim sigur  $P$ , un generator  $G$  al subgrupului de ordin  $Q$ , iar protocolul parcurge patru etape: generarea cheilor, unde cheia publică  $y = G^x \bmod P$  este stocată de server; angajamentul, unde clientul transmite  $t = G^r \bmod P$ ; provocarea, unde serverul generează aleatoriu  $c$ ; și răspunsul, unde clientul calculează  $s = (r + c * x) \bmod Q$ .

Verificarea constă în evaluarea egalității  $G^s = t * y^c \bmod P$ . Dacă este satisfăcută, identitatea este confirmată matematic, fără ca secretul să fi fost expus.

#### Slide 6 (Maparea Schnorr peste OAuth 2.0)

Fazele protocolului Schnorr se mapează direct peste fluxul OAuth 2.0. Angajamentul servește drept inițiere a cererii de acces, provocarea funcționează ca nonce de sesiune, răspunsul matematic înlocuiește parametrul clasic `client_secret`, iar la finalizare serverul emite un jeton JWT.

Diferența fundamentală: în protocolul propus, niciun material secret nu traversează rețeaua, în timp ce în toate variantele OAuth clasice, parola este transmisă în faza de autentificare.

#### Slide 7 (Tehnologii)

Python cu Flask a fost ales pentru serverul de autentificare datorită ecosistemului extins de biblioteci criptografice. Flask-SQLAlchemy asigură persistența într-o bază SQLite locală. Am utilizat `cryptography` și `sympy` pentru parametrii criptografici, `PyJWT` pentru jetoane, iar `Authlib` pentru implementarea de referință OAuth 2.0 PKCE.

Suita de testare se bazează pe `pytest` pentru cazurile funcționale și vectorii de atac, iar `Locust` pentru evaluarea performanței sub încărcare.

JavaScript a fost ales pentru client, unde toate calculele criptografice se execută local, prin `BigInt` și `Web Crypto`, fără cadre de lucru externe.

#### Slide 8 (Arhitectură)

Arhitectura este de tip trei niveluri. Nivelul de prezentare cuprinde interfața web statică unde se execută calculele criptografice ZKP pe dispozitivul utilizatorului.

Nivelul de aplicație, conturat de Flask, gestionează logica criptografică, emiterea jetoanelor JWT și modulele comparative OAuth 2.0.

Nivelul de date conține persistența într-o bază SQLite cu trei tabele: utilizatorii cu cheile publice, jetoanele emise și datele protejate. La nivelul bazei de date nu se stochează niciun secret privat.

### Slide 9 (Fluxul de înregistrare)

În procesul de înregistrare, clientul derivă local cheia privată din parolă și calculează cheia publică  $y = G^x \bmod P$ . Către server se transmite exclusiv cheia publică, care este stocată și asociată identității utilizatorului.

Parola și cheia privată nu părăsesc niciodată dispozitivul clientului. În eventualitatea compromiterii bazei de date, valorile publice sunt inutile criptografic, derivarea cheii private fiind echivalentă cu rezolvarea problemei logaritmului discret.

### Slide 10 (Fluxul de autentificare)

Clientul generează un nonce aleatoriu, calculează angajamentul  $t$  și îl transmite. Serverul creează o sesiune temporară cu un timp de viață de 5 secunde și returnează provocarea aleatorie  $c$ .

Clientul recalculează cheia privată din parolă, calculează răspunsul  $s = (r + c * x) \bmod Q$  și îl transmite. Serverul verifică egalitatea matematică utilizând exclusiv cheia publică stocată și, la succes, emite un jeton JWT ancorat de contextul sesiunii ZKP.

### Slide 11 (Proprietăți de securitate)

Rezistența la interceptare: un adversar vizualizează exclusiv transcrisul efemer, ecuația  $s = (r + c * x) \bmod Q$  neputând fi rezolvată fără nonce-ul  $r$ , care nu este transmis.

Eliminarea secretului partajat: serverul stochează doar cheia publică, neutralizând impactul breșelor de date. Provocarea unică per sesiune previne atacurile replay. Legarea de canal și expirarea JWT atenuează deturnarea sesiunii.

Arhitectura se înscrie în paradigma identității auto-suverane, validând identitatea prin rigoare matematică, fără furnizori terți de încredere.

### Slide 12 (Validare și teste de securitate)

Validarea a fost realizată prin suite pytest acoperind cazuri pozitive, negative, scenarii limită și vectori de atac. Ecuația de verificare este satisfăcută în toate scenariile cu credențiale valide și respinsă la tentative de falsificare.

Auditul de trafic a confirmat că niciun mesaj HTTP nu conține parola brută sau derivate directe. Atacurile simulate de tip replay, session hijacking, compromitere de date și simulator attacks au fost respinse cu succes.

### Slide 13 (Evaluarea performanței)

Evaluările au indicat o latență a verificării criptografice de aproximativ 1,12 milisecunde per operație și un debit susținut de circa 170 de cereri pe secundă. Compromisul temporal față de schemele clasice este justificat de garanțiile criptografice superioare obținute.

Protocolul se încadrează în parametrii optimi pentru autentificarea interactivă, latența totală rămânând imperceptibilă pentru utilizatorul final.

#### Slide 14 (Comparație ZKP vs. OAuth 2.0)

În protocolul ZKP Schnorr, secretul nu traversează niciodată rețeaua. În fluxurile OAuth PKCE, simplificat și Authlib, parola este transmisă în faza de autentificare, chiar dacă restul fluxului beneficiază de protecție.

Din perspectiva costurilor, fluxul ZKP introduce o latență ușor superioară, însă acest compromis este justificat de eliminarea completă a riscului de exfiltrare a parolei.

#### Slide 15 (Concluzii)

Implementarea demonstrează viabilitatea autentificării web fără transmiterea parolei către server. Protocolul satisface rigurile funcționale, de securitate și de performanță, iar integrarea în ecosistemul OAuth confirmă compatibilitatea cu standardele existente.

Ca limitări, parametrii actuali, precum  $G = 4$  și  $\text{scrypt } n = 2^{11}$ , sunt adecvați exclusiv prototipului. Ca direcții viitoare, se impune tranziția către curbe eliptice, derivarea cheilor prin Argon2, externalizarea sesiunilor către Redis, semnături asimetrice pentru jetoane și implementarea obligatorie a HTTPS.

#### Slide 16 (Bibliografie selectivă)

Acestea sunt câteva referințe din bibliografia utilizată.

#### Slide 17 (Mulțumesc)

Vă mulțumesc pentru atenție!

Textul total este de aproximativ 1100 de cuvinte, calibrat similar cu template-ul de licență (aproximativ 1000-1200 cuvinte), potrivit pentru o susținere de 8-10 minute la un ritm normal de vorbire.